PES UNIVERSITY, Bangalore

Established under Karnataka Act No. 16 of 2013)

Department of Computer Science & Engineering

UE24CS251A: DIGITAL DESIGN AND COMPUTER ORGANIZATION

Unit 1:

1. *Using Boolean laws, prove that the expression* F = AB + A'C + BC *can be simplified to* F = AB + A'C.
   Explain the rationale behind removing the redundant term and the Boolean theorem used.
2. A student proposes a circuit for F = AB + A'C but implements it as (A + B)·(A' + C). Is the implementation logically correct? Justify your answer with a truth table comparison and Boolean reasoning.
3. You are given a Boolean function implemented in **3 logic levels** as F=AB+C(D+E)F = AB + C(D + E)F=AB+C(D+E)

   a) Why is this not a standard SOP or POS form?
   b) Convert this expression into a **canonical SOP form**.
   c) Discuss the impact of logic levels on gate delay and suggest why a two-level form is preferred in digital design.

4. In digital logic design, it is often desirable to minimize Boolean functions.
   a) What is **logic minimization**, and why is it important in hardware design?
   b) What metric is commonly used to measure the efficiency of a minimized Boolean function?
   c) Explain how canonical forms help in the process of logic minimization, and state one limitation of using them directly for circuit implementation.
5. Design a 3-variable logic circuit whose output is 1 for minterms 1, 3, 5, and 6. Draw the K-map, minimize the function, and draw the logic circuit diagram using basic gates.

6. Design a **4-variable logic circuit** whose output is **1 for minterms 0, 1, 2, 5, 7, 8, 10**. The output is **don't care (X)** for minterms **3 and 11**.

   a) Draw the **4-variable Karnaugh Map**.

   b) Minimize the function using **SOP form** (use don't-cares to simplify).

   c) Minimize the same function using **POS form** (also using don't-cares).

   d) Compare the SOP and POS expressions in terms of **gate complexity**.

   e) Draw the logic circuit diagram for the **simplified SOP expression** using basic gates

7. Implement the Boolean function **F = AB + CD** using:
(a) AND-OR logic (standard SOP)
(b) **Only NAND gates**
Draw both circuits and explain the conversion process from SOP to NAND implementation.

8. Describe the steps involved in converting a Boolean function implemented using **AND-OR logic** to an implementation using **only NOR gates**.
Using this method, convert and implement **F = (A + B)(C + D)** using NOR gates only.

9. Design a combinational circuit that converts a 4-bit **BCD (8421)** input into **Excess-3** code. Your design should include:

- (a) A complete truth table (consider valid BCD inputs only)
- (b) Boolean expressions for each output bit
- (c) Simplification using Karnaugh Maps (K-maps)
- (d) Final logic diagram using basic gates (AND, OR, NOT)

10. A combinational circuit has the following gate-level structure:

- Inputs: A, B, C
- Gate 1: Output X = A $\oplus$ B
- Gate 2: Output Y = X · C
- Gate 3: Output Z = Y + A'

Perform the following:

- (a) Derive the Boolean expression for the final output Z
- (b) Simplify the expression using Boolean algebra
- (c) Draw the logic circuit diagram corresponding to the simplified expression

11. Design a **binary adder-subtractor** circuit that uses a common logic to perform both operations.

- Explain how the 2's complement is used in subtraction.
- Show circuit block with controlled inverter using XOR.

12. How is **overflow** detected in binary addition for both **signed** and **unsigned** numbers?

- Provide the condition and example for each case.

13. What is the need for a **BCD adder** instead of a binary adder for decimal digit operations?

- Describe the correction logic required when binary sum exceeds 9.
- Derive the correction condition using Boolean logic.

14. Draw the block diagram of a **BCD adder** that adds two BCD digits with a carry-in.

- Explain the role of upper and lower 4-bit adders and how the correction is applied.

15. Explain how a 2-bit binary multiplier works. Draw the circuit and explain the role of AND gates and adders in producing the final product.

16. Construct the truth table for a 2-bit magnitude comparator and explain how the circuit distinguishes between A > B, A = B, and A < B conditions.

17. Design a 3-to-8 decoder using only 2-to-4 decoders and logic gates. Explain the logic behind the construction.

18. Explain how complemented minterm expressions can help reduce hardware complexity when implementing functions using a decoder.

- Provide an example where this strategy reduces the number of OR gate inputs.

19. Design a 4-to-1 multiplexer using only 2-to-1 multiplexers. Show the logic circuit and explain the design.

20. Compare the implementation of Boolean functions using multiplexers vs. decoders. Highlight the trade-offs involved.